Assignment 6: Build, Measure, and Explain Locks, Condition Variables, and Deadlocks

Linux Systems and Performance - Week 6

Bounded Queue -> Shorter Lock Paths for a UI Thread -> Diagnosing and Fixing Deadlocks

1. Assignment Philosophy

This assignment is not evaluated only by whether the code runs. You will build a bounded queue with a mutex and condition variables, use it to shorten the lock path of a UI thread in a small album service, and diagnose a service that hangs with lock checking tools, then fix it by changing the order or the kind of its locks. You will check each of them with the provided tests, kernel events, and the Week 6 concurrency model.

Each level pairs code you write with a provided test or tracing tool. When the numbers your program reports and the numbers a tool reports for the same run disagree, explain why. Make it correct first, then measure what each change cost and what it saved.

PREDICT -> BUILD -> MEASURE -> ANALYZE -> EXPLAIN

Important: AI coding tools may be used. However, you are responsible for understanding, debugging, modifying, and explaining all submitted code and experimental results. Implementation alone is not sufficient for full credit.

2. Challenge Levels

Level	Build	Load / Trigger	Measure / Observe	Main Question
Level 1	bounded queue (bq): put, get, timed get, close	producers and consumers, capacity, signal vs. broadcast	lost or duplicate items, sleeps, futex calls by op and caller, TSan	Why a while loop, and why must close wake everyone?
Level 2	snapshot and handoff modes of albumsvc	save under the lock, slow storage, a CPU hog next to the saver	frame miss and p99, lock call latency, the owner and what it does	How do you show that a UI delay comes from a lock, and what shortens it?
Level 3	diagnose and fix the four deadlocks of devsvc	cases 1 to 4, deliberately wrong fixes, a writer-preferring rwlock	lockcheck W1 to W4, TSan reports, HANG, parallel and ops/s	Did the fix remove the deadlock, or only hide the symptom?

Level 2 handoff uses the Level 1 queue. If Level 1 is not finished, build albumsvc with the provided lib/libbq_ref.a (make albumsvc_ref, then ALBUMSVC=./bin/albumsvc_ref bash tools/l2_run.sh ...) and say so in your report. Absolute performance across different machines is not graded. Kernel modification is not required.

3. Common Interface and Rules

- Use C for the required code unless otherwise approved. Run on your laptop with WSL2 kernel 6.6, two vCPUs (processors=2), and the bpftrace installed in Lab 5. Build with the provided Makefile, which uses -g -fno-omit-frame-pointer and exports function names with -rdynamic.
- Fixed parameters: albumsvc draws a frame every 16667 us with 2000 us of drawing work, the album has 256 items, two scanner workers spend 3000 us "reading a file" per update, and the saver saves every 240 ms with 3000 us of compression work and 20 ms of slow storage after fsync. In handoff mode ui_inbox holds 64 updates, saver_inbox one snapshot, and the UI applies at most 64 updates per frame. The checks of bqtest and devsvc, the load of devsvc (the WORK_* values in src/devsvc.h, four workers, a hang after 3 s), and the tool scripts are fixed as well. Everyone uses the same values, so results are comparable across reports.
- Do not modify src/bqtest.c, src/common.h, src/bq.h, src/album.h, src/devsvc.c, src/devsvc.h, src/rcu.c, src/rcu.h, tools/, or lib/. Keep the names and signatures of the TODO functions. In the Level 3 case files, change only the TODO(L3-n) regions, and do not remove a svc_work call or move it out of the lock section it is in now.
- Measure time only with the normal build. Use the TSan builds (bin/bqtest_tsan, bin/devsvc_tsan) only to check for races and lock misuse, because the instrumentation slows the program down. Never load lockcheck.so into a TSan binary.
- Predict before running each experiment. Repeat measurements. Record exact commands, PID, uname -r, and execution time in raw logs saved with script -c or tee. Every measurement row must identify its source log. Keep bpftrace output as the original file.
- A futex call is not proof of mutex contention, and a long wait in perf sched is not proof of a lock bottleneck. When you name a cause, support it with at least two kinds of evidence, for example the futex op together with the calling function, or the lock call latency together with what the owner was doing.
- Document kernel- and glibc-version-dependent details when they affect the implementation or observed behavior. Screenshots alone are not sufficient. Include short interpretations and cite your own measurements.

4. Required Interface and Test Tools

```
./bin/bqtest [-p producers] [-c consumers] [-n items] [-q capacity] [-b] [-w work_us] [-T sec]
./bin/bqtest close
./bin/bqtest timeout
bash tools/l1_check.sh
sudo bpftrace tools/futex.bt -c './bin/bqtest ...'
./bin/albumsvc [-m lock|snapshot|handoff] [-d seconds] [-D disk_ms] [-Z compress_us]
               [-S save_ms] [-c saver_cpu] [-o frames.csv]
bash tools/l2_run.sh <mode> [albumsvc options]
sudo bpftrace tools/lockwait.bt <PID> <seconds>
bash tools/burn.sh <cpu> [seconds]
./bin/devsvc -c <1|2|3|4> [-t threads] [-d seconds]
LD_PRELOAD=./bin/lockcheck.so <program> ...
bash tools/l3_run.sh <1|2|3|4> [devsvc options]
bash tools/l3_check.sh <1|2|3|4>
```

The skeleton (Assignment6_skeleton_code.zip) builds and runs as it is. The Level 1 and 2 TODO functions are stubs, so their tests fail until you implement them. The Level 3 case files build and run, but soon hang (deadlock). l2_run.sh asks for the sudo password once, so type one command at a time. If a TSan binary stops with "unexpected memory mapping", run sudo sysctl vm.mmap_rnd_bits=28.

File	What You Implement
src/bq.c	TODO(L1-1 to 4) bq_put, bq_get, bq_timed_get, bq_close
src/album.c	TODO(L2-1) album_snapshot, album_snapshot_free
src/albumsvc.c	TODO(L2-2 to 5) snapshot saver, handoff send, apply, and save
src/sink.c	TODO(L3-1) case 1: the order of the two sink locks
src/bus.c	TODO(L3-2) case 2: callbacks that call back into the bus
src/catalog.c	TODO(L3-3) case 3: rwlocks and their modes
src/config.c	TODO(L3-4) case 4: the config read with RCU, and its grace period

5. Level 1 - Bounded Queue

5.1 Required Implementation

- bq.h is the contract. bq_put blocks while the queue is full, bq_get while it is empty. Both return BQ_OK on success and BQ_CLOSED when the queue is closed (bq_get only after the remaining items are taken). Write every wait loop with BQ_WHILE instead of while (see bq.h) and count each sleep in n_put_waits or n_get_waits.
- bq_timed_get(q, &item, timeout_ms): a relative time of 0 or more ms on CLOCK_MONOTONIC. Compute the deadline once on entry, so wake-ups without an item do not extend it. 0 never blocks. If several conditions hold, an available item gives BQ_OK, otherwise a closed queue gives BQ_CLOSED, otherwise BQ_TIMEOUT.
- bq_close marks the queue closed and wakes every waiting producer and consumer, whatever BQ_BROADCAST says. A put that is waiting or comes later returns BQ_CLOSED without enqueuing. A second bq_close has no effect. bq_destroy may be called only after every thread that used the queue has returned from all bq calls.
- bqtest checks that every item arrives exactly once, and the dequeue order: each consumer must see the items of one producer in increasing sequence (with one consumer, exactly the producer order). With -w, consumers spend work_us on each item, and the order in which they finish is printed as completion_inversions for information only. It is not a FIFO violation.

5.2 Required Experiments

```
bash tools/l1_check.sh
for q in 1 4 64; do ./bin/bqtest -p 2 -c 2 -n 100000 -q $q | tee logs/l1-q$q.txt; done
./bin/bqtest -p 4 -c 4 -n 50000 -q 4    | tee logs/l1-signal.txt
./bin/bqtest -p 4 -c 4 -n 50000 -q 4 -b | tee logs/l1-broadcast.txt
./bin/bqtest -p 2 -c 2 -n 50000 -q 4 -w 20 | tee logs/l1-work.txt
for q in 1 64; do
  sudo bpftrace tools/futex.bt -c "./bin/bqtest -p 2 -c 2 -n 20000 -q $q" > logs/l1-futex-q$q.txt
done
make bugif && ./bin/bqtest_bugif -p 2 -c 2 -n 20000 -q 4 -T 10 2>&1 | tee logs/l1-bugif.txt
```

Report a compact comparison table. Add the exact configuration and source-log path to each row.

Comparison	Prediction	items/s, put/get waits	futex by op and caller (CAP 1, 64)	Interpretation
CAP 1 / 4 / 64				
signal / broadcast				
correct build / BUG_IF				

5.3 Required Explanation

1. Show which failures the BUG_IF build produced (lost, duplicate, order_violation, early_closed, or a hang). Explain one of them with an interleaving of two threads, and why the same code passes in the correct build.

2. In the default signal mode, why must bq_close still wake every waiter? Which close test would fail if bq_close used signal, and which threads would stay asleep? Why must closed be checked under the same mutex as the wait (the lost wake-up of the lecture)?

3. Compare items/s and put/get waits for CAP 1, 4, and 64. With futex.bt, separate the futex calls made inside pthread_mutex_lock, pthread_mutex_unlock, pthread_cond_wait, pthread_cond_signal, and pthread_join. Which of them would remain without any lock contention? Explain why many futex calls do not by themselves mean mutex contention.

4. With -w, completion_inversions is nonzero while the dequeue order check passes. Explain the difference between dequeue order and completion order, and what a FIFO queue does and does not promise to consumers. Then explain why bq_timed_get computes its deadline once on CLOCK_MONOTONIC, and what the deadline_fixed test would show otherwise.

6. Level 2 - Lock Delay on a UI Thread

6.1 Required Implementation

- The UI thread of albumsvc starts frame k at S_k = S_0 + k x 16667 us, reads the whole album, then draws for 2000 us. Frame latency (frame_late in the output) is E_k - S_k, where E_k is the time the frame finished. A miss is a frame with E_k > S_k + 16667 us, and p99 is taken over the latency of all drawn frames. After a miss the UI resumes at the next scheduled start and counts the starts it skipped. ui_lock is the time from calling pthread_mutex_lock to its return in the UI thread: the latency of the call, not pure waiting time.
- Two scanner workers change items, and a saver saves the album every 240 ms. In lock mode (provided) the saver holds album_lock through serializing, compression, write, fsync, and the slow storage delay. The goal is to reduce the UI delay caused by the lock and to show the effect by measurement, not to remove every late frame.
- snapshot mode (TODO L2-1, L2-2): album_snapshot makes a deep copy, titles included, because each title is a heap string that a worker may free after the saver unlocks. The snapshot owns all of its memory, and the saver frees it after saving. No pointer into the live album may survive the unlock. The saver holds album_lock only while copying.
- handoff mode (TODO L2-3 to 5): the UI thread is the only owner of the album, and the album has no lock. Workers send each update through a bq (ui_inbox) and never touch it again. Each frame the UI applies queued updates without blocking, up to a budget. When a save is due, the UI makes the snapshot and puts it into saver_inbox, and the saver writes the file.
- lockwait.bt (provided) measures pthread_mutex_lock call latency per thread, the thread that owned the mutex when a slow call began and the function that made the call, and the write, fsync, and sleep system calls made while holding a mutex.

6.2 Required Experiments

```
for m in lock snapshot handoff; do ./bin/albumsvc -m $m | tee logs/l2-$m-notrace.txt; done
bash tools/l2_run.sh lock
bash tools/l2_run.sh snapshot
bash tools/l2_run.sh handoff
TAG=lock-D5 bash tools/l2_run.sh lock -D 5
TAG=lock-cpu bash tools/l2_run.sh lock -D 0 -c 1
bash tools/burn.sh 1 30 &
TAG=lock-burn bash tools/l2_run.sh lock -D 0 -c 1
bash tools/burn.sh 1 30 &
TAG=snapshot-burn bash tools/l2_run.sh snapshot -D 0 -c 1
```

l2_run.sh runs albumsvc for 10 s, attaches lockwait.bt for 8 s, and records 2 s of perf sched for the ui and saver threads. TAG names the log files. -D 0 -c 1 removes the storage delay, so the lock is held mainly for the compression work plus write and fsync, and pins the saver to CPU 1, where burn.sh runs. Report a compact comparison table. Add the exact configuration and source-log path to each row.

Configuration	Prediction	miss, frame p99	ui_lock or ui_apply p99	saver_hold, lockwait evidence	Interpretation
lock / snapshot / handoff					
lock -D 5					
-D 0 -c 1: lock, lock + burn, snapshot + burn					

6.3 Required Explanation

5. For lock mode, quote miss, frame latency p99, ui_lock p99, and saver_hold. Use lockwait.bt to show which thread owned album_lock during the UI's slow calls and which system calls it made while holding it. Compare ui_lock of the run without tracing with ui_lock and @lock_call_us[ui] of the traced run, and explain the differences (the uprobes cost something too).

6. For snapshot mode, state exactly what album_snapshot copies and why copying only the title pointers is not enough. Describe what would fail with a shallow copy. How did saver_hold and the UI numbers change, and what did the change cost, and where?

7. For handoff mode, name the owner of the album, who builds the save snapshot, and who writes the file. What replaced ui_lock, and what does the UI pay now? Quote ui_apply from the run without tracing and explain why it grows under lockwait.bt. Reason without a new run: what would happen to frame latency without the apply budget if many updates were queued?

8. In perf sched timehist the ui thread shows a long wait time in every mode. Why? Why can a long wait with a short scheduling delay alone not prove a lock bottleneck? Which evidence from lockwait.bt (the caller, lock call latency, the owner, and what the owner did) did you use for your conclusion?

9. Compare lock -D 0 -c 1 with and without burn.sh, and snapshot under the same load. Explain with the Week 5 scheduling model why saver_hold and ui_lock grow although the saver's work did not change (lock holder preemption), and why snapshot is barely affected.

7. Level 3 - Diagnosing and Fixing Deadlocks

7.1 Required Implementation

- devsvc is a small device service made of four cases. Each case is one file, and ./bin/devsvc -c N sends requests to that case only, from four worker threads, then checks the results. The case files of the skeleton hang when you run them. Change only the TODO(L3-n) region of each file. The driver (src/devsvc.c), src/devsvc.h, src/rcu.h, src/rcu.c, and tools/lockcheck.c are provided.
- There are three diagnostic tools. lockcheck.so (provided) is loaded with LD_PRELOAD and reports W1 to W4. W1 is a possible deadlock (a cycle in the lock order graph, or taking again a lock the thread holds), W2 a lock still held at exit (also when a run ends with HANG), W3 a call that may sleep (read, write, fsync, the sleep functions) while holding a lock, W4 an unlock of a lock the thread does not hold. It handles mutexes and rwlocks, keeps the read or write mode of every edge, and reports W1 only for a cycle in which every lock really makes a thread wait. Taking a recursive mutex again is not a warning. The TSan build (bin/devsvc_tsan) reports data races and the lock order problems that TSan finds on its own. The driver ends a run with result: HANG and prints what every worker was doing when a worker finishes no request for 3 s.

Each case restricts how it may be fixed. A fix that breaks the rule does not count, even if it passes l3_check.sh.

Case	File and Content	Rule for the Fix
1	src/sink.c: moves a session from one sink to another. Every sink has a mutex	Keep one mutex per sink and the lock type. Fix the order in which the locks are taken
2	src/bus.c: an event bus. Callbacks call bus functions again	No recursive mutex. Make it safe for a callback to call back into the bus
3	src/catalog.c: finds the permissions of an app and the apps of a permission. Most requests only read	Change the two locks to pthread_rwlock_t and take each in the mode the path needs. Keep the lock order of the two read-only paths. Initialize them with svc_rwlock_init
4	src/config.c: every frame reads the picture config. Updates are rare	Readers of the config take no cfg_lock and use rcu.h. Free an old config only after a grace period. Keep the output locks

- tools/l3_check.sh checks five criteria for one case. (1) lockcheck: no warning at all and result: PASS. (2) TSan: no report and result: PASS. TSan's lock order check is off here (detect_deadlocks=0), so it looks for data races only, and lock order is left to lockcheck. (3) stress: five runs of 2 s with -t 2, 4, 4, 4, 8, all PASS. (4) parallel: at least 3% in every -t 4 run. (5) speed: the three -t 4 runs alternate with the reference fixes (bin/devsvc_ref), and the median ops/s must be at least 0.70 x the reference median.
- The driver's checks differ by case. Case 1 counts all sessions, case 2 checks that every persistent subscriber was called as many times as there were events, case 3 that no app holds a disabled permission, and case 4 that every frame read a consistent config and that the last version was applied to every output. parallel is the share of the work inside the case (svc_work calls) that found another svc_work call running when it started or ended. A fix that lets only one request work at a time gives 0%.

7.2 Required Experiments

```
for c in 1 2 3 4; do TAG=$c-before bash tools/l3_run.sh $c; done   # before any fix
bash tools/l3_run.sh <case>            # after the fix, for every case
bash tools/l3_check.sh <case>
TAG=w2 bash tools/l3_check.sh 2        # wrong fixes: change only what is described below, then undo it
TAG=w3 bash tools/l3_check.sh 3
TAG=w4a bash tools/l3_check.sh 4
TAG=w4b bash tools/l3_check.sh 4
DEVSVC_RWLOCK=writer TAG=3-writer bash tools/l3_run.sh 3
```

For a wrong fix, change only the following in your own fix and run it. w2: make bus_lock PTHREAD_MUTEX_RECURSIVE and put the original bus_publish back. w3: keep the rwlocks and modes, but make one writing path (catalog_set_perm or catalog_grant) take the two locks in the order opposite to the other writing path. w4a: move synchronize_rcu and config_free inside cfg_lock. w4b: remove synchronize_rcu and free at once. Show the changed lines in the report. DEVSVC_RWLOCK=writer makes svc_rwlock_init create writer-preferring rwlocks (PTHREAD_RWLOCK_PREFER_WRITER_NONRECURSIVE_NP). Report a compact comparison table. Add the exact configuration and source-log path to each row.

Run	Prediction	lockcheck: Warnings, Functions, Locks, Modes	TSan Reports	result, parallel, ops/s	Interpretation
before the fix: case 1 / 2 / 3 / 4					
after the fix: case 1 / 2 / 3 / 4					
w2 / w3 / w4a / w4b					
case 3 with a writer-preferring rwlock					

7.3 Required Analysis

10. For the four cases before the fix, name the paths that form each cycle from the W1 output (the functions, locks, and modes on the now and earlier lines, case 2 is a one-line warning), and match them with what the workers were doing in the HANG output. Explain from how each tool models locks why TSan reported nothing for case 2, and why the TSan run of l3_run.sh reports a lock-order-inversion for the fixed case 3.

11. In w2 there is no W1 and no HANG. Which check fails? Explain how it fails from the loop of bus_publish and the way bus_unsubscribe fills the hole, and state which invariant is broken while a callback runs. How does your fix avoid it, and what more would it need if a struct sub could be freed? Finally compare parallel and ops/s of w2 with the reference, and explain why on two vCPUs the difference in ops/s is smaller than the difference in parallel.

12. Why do the two read-only paths of case 3 hang with mutexes but not in read mode? Quote the W1 output of w3 with its modes to show that edges that are not read-read still form a cycle, and state the rule the writing paths must follow. From the run with a writer-preferring rwlock, explain why the same code becomes unsafe again. How do the Linux kernel's rwsem and lockdep handle this case?

13. In case 4, why would changing cfg_lock to an rwlock alone still hang? Explain with the modes (you may reason without a run). From the lockcheck report and the HANG output of w4a, state who waits for whom, and connect it to why a thread that waits for a lock cannot pass a quiescent state in QSBR. What did which tool or check report for w4b? Also explain why a reader in your fix must load cfg only once.

8. Evidence Required

Level	Implementation Evidence	Runtime / Measurement Evidence	Analysis Evidence
1	bq with the close and timeout contract, l1_check PASS in normal and TSan builds	capacity and wake comparisons, futex by op and caller, BUG_IF failures	while vs. if, lost wake-up, futex vs. contention, dequeue vs. completion order
2	deep-copy snapshot, handoff with clear ownership	three modes, -D 5, burn.sh with -c 1, lockwait.bt, timehist	lock call latency vs. waiting, owner activity, lock holder preemption
3	fixes of the four cases that keep the case rules, all five l3_check criteria PASS	l3_run before the fixes, wrong fixes w2, w3, w4a, w4b, writer-preferring rwlock	lock order and modes, hiding a symptom vs. fixing the cause, what each tool can and cannot see

9. Submission Package

- src/bq.c, src/album.c, src/albumsvc.c, src/sink.c, src/bus.c, src/catalog.c, src/config.c, any analysis scripts you wrote, and the original log directory.
- README with kernel version, glibc version, vCPU count, bpftrace version, exact build/run commands, and highest completed level.
- Technical report (recommended 4-6 pages) organized as Level 1 / Level 2 / Level 3. Include predictions, controlled comparisons, measured results, and explanations.
- Level 1 capacity and wake tables, Level 2 mode tables with lockwait evidence, and the Level 3 case table before and after the fixes, with the wrong fixes.
- Selected runtime evidence and exact commands. Every measurement row must identify its raw log. Do not submit pages of unannotated raw output.
- A short AI-use statement if AI tools were used, including at least one generated suggestion or code issue that you verified, changed, or rejected.

10. Evaluation

Category	Weight	What Is Evaluated
Implementation	25%	a queue that passes the close, timeout, and stress tests in normal and TSan builds, snapshot and handoff modes with clear ownership, and devsvc fixes that keep the case rules and pass l3_check
Experiment & Measurement	35%	predictions, fixed parameters, PID/kernel logs, original bpftrace output, separate timing and TSan builds, and repeatability
Technical Explanation	30%	reasoning from measured values about condition variables and lost wake-ups, futex vs. contention, lock call latency vs. scheduling, lock holder preemption, lock order and modes, the RCU grace period, and what each checking tool can see
Code Quality & Report	10%	check-off modifications, reproducibility, and concise evidence

Performance is not graded by absolute numbers across different hardware. The quality of the controlled comparison, the correctness of the measurement method, and the explanation of observed behavior are graded.

A program that appears to work but is not experimentally validated or correctly explained cannot receive full credit.

11. Final Check-Off Questions

The oral check-off takes approximately five minutes. Predict each result before making the change and running it.

- Make bq_get return BQ_CLOSED as soon as the queue is closed, without draining. Predict which bqtest checks fail and why.
- Run albumsvc in lock mode with -D 40. Predict miss and skipped before running, and explain why they differ.
- Make catalog_set_perm of your fix take both locks in write mode. Predict what lockcheck reports, and explain with the modes which two paths form the cycle.
- Run snapshot mode with -S 50 (a save every 50 ms). Predict how ui_lock, saver_hold, and saver_cpu_ms change before showing the measurement.

Build it. Measure what changes. Explain why.